

02 世界模型是怎么工作的？

作者 NILUSHANAN KULASINGHAM

阅读约 25 分钟 · 可交互

一个能往前推演的模型，让你在真的付出代价之前先把一个动作试一遍。这是这个领域最老的想法，也仍然是最好的那个。麻烦在于，一个强到能找出最佳方案的搜索，也强到能找出模型出错的地方，而这一点至今没有人从工程上消掉。

本章的图都是可交互的。请到 worldmodels101.com/zh/chapters/the-idea 在线按一按、拖一拖。

我想从一个路口讲起，因为我觉得它就是最普通不过的那个世界模型。你在支路上等着右转。你往右边看，有一辆车正开过来。接下来这一秒里的某个时刻，你要做个决定：现在就走，还是等它过去。

你在那一秒里做的事，就是把这个场景往前推演一遍。那辆车在那儿，大概开那么快，所以两秒之后它会到这里。我现在走的话能避开它，我等着的话也不挡它的道。什么都没发生，你就凭自己对它的预测把决定定下来了。

大多数时候这个预测够用，整件事花掉的时间比你读完它还短。不够用的那些情形才是有意思的。新手司机会看错来车的速度，而在这里预测错了，收场就是一次你没法撤销的碰撞。我们把这个决定做成一个你可以自己跑一遍的东西。

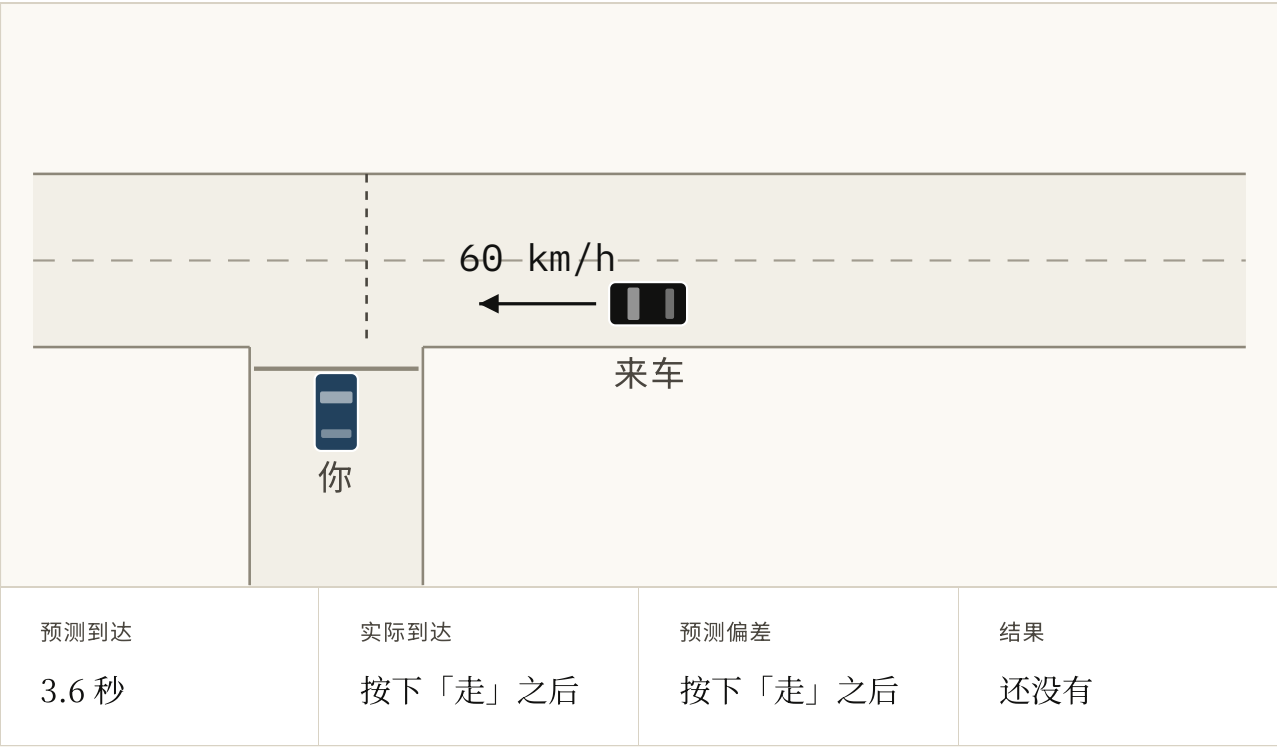


FIG. 2.1 把预测摆到明面上的那个路口。设一下来车离你多远、开得多快，再读一下预测怎么说。先保持在新手司机上，按一下「走」，看看你过不过得去。然后切到有经验的司机，同样的间隙再跑一遍。数字仅作示意。

你也看到了，从你做预测到揭晓答案，来车什么都没变。变的只是你对它的预测有多准，而这决定了你是过去了还是撞上了。过了某个速度以后，做得好不好，取决于还没发生的事情。

拿上你手里有的，往前推演，朝着接下来会发生的地方移动。这就是动力学模型。在 *世界模型* 这个说法后来被扯着盖住的所有东西里，第 1 章数出了五种意思，而它是最老的一个，也是这个词当初被造出来时指的那个。造词发生在 1990 年，第 1 章讲过那段故事。

模型本身只预测会发生什么。读这些预测并做出选择的是另一个东西，那个东西有自己的名字：**策略**（选出动作的那一部分，不管它是靠使劲想还是靠纯反射来选）。Jürgen Schmidhuber 1990 年的那个版本，就把这两者当成两个分开的对象，而这个区分在这一章里从头贯到尾。

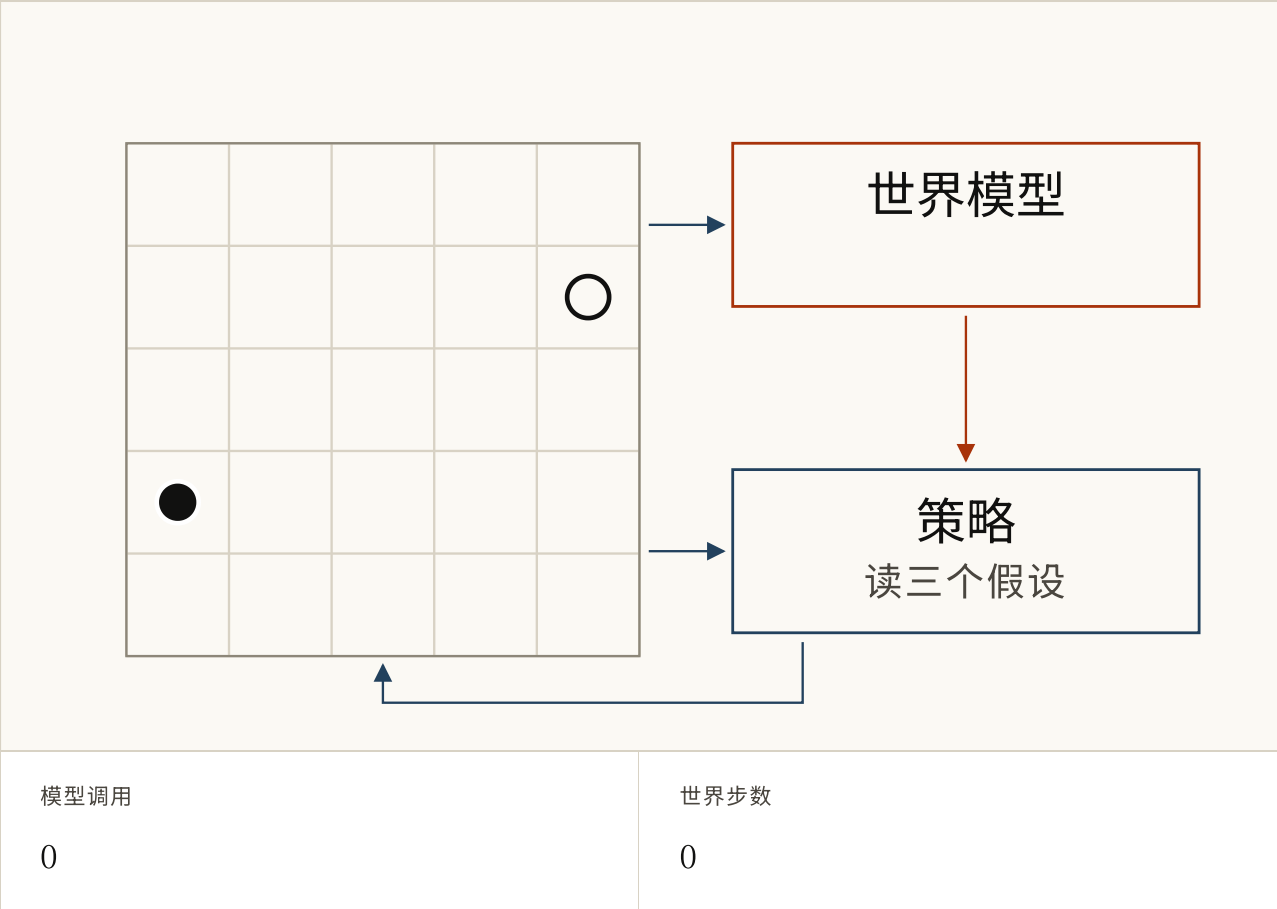


FIG. 2.2 拆开来画的两个半边。按一下「问模型」，它就回答三个「如果」，一个动作一个，全程没碰到世界。按一下「行动」，策略从里面挑一个，世界往前走一步。现在把模型关掉，再按一次「行动」：策略照样行动，直接按它看到的来，什么也没想象过。只有策略会碰到世界。仅作参考。

所以下次有人告诉你他们的系统是个世界模型，你要问的是他们指的是哪一半。第一半能换来很多东西，同时它身上也有一种再多工程也没能去掉的失败方式。

模型装了什么，又可以丢掉什么

它的形状从 1990 年起就没变过：你在哪里，你做了什么，接下来是什么。动力学模型学的就是这个映射 (dynamics 就是事物如何随时间变化，所以这是一个关于「接下来会发生什么」的模型)。它通常还会顺带学「这一步值多少」。把它写下来，头上带顶帽子，长这个样子。

$$\hat{p}(\underline{z}_{t+1}, \underline{r}_t \mid \underline{z}_t, \underline{a}_t)$$

在 给 定 你现在在哪里 和 你可能采取的一个动作 的条件下，该期待你会落到哪里以及这一步值多少是什么。

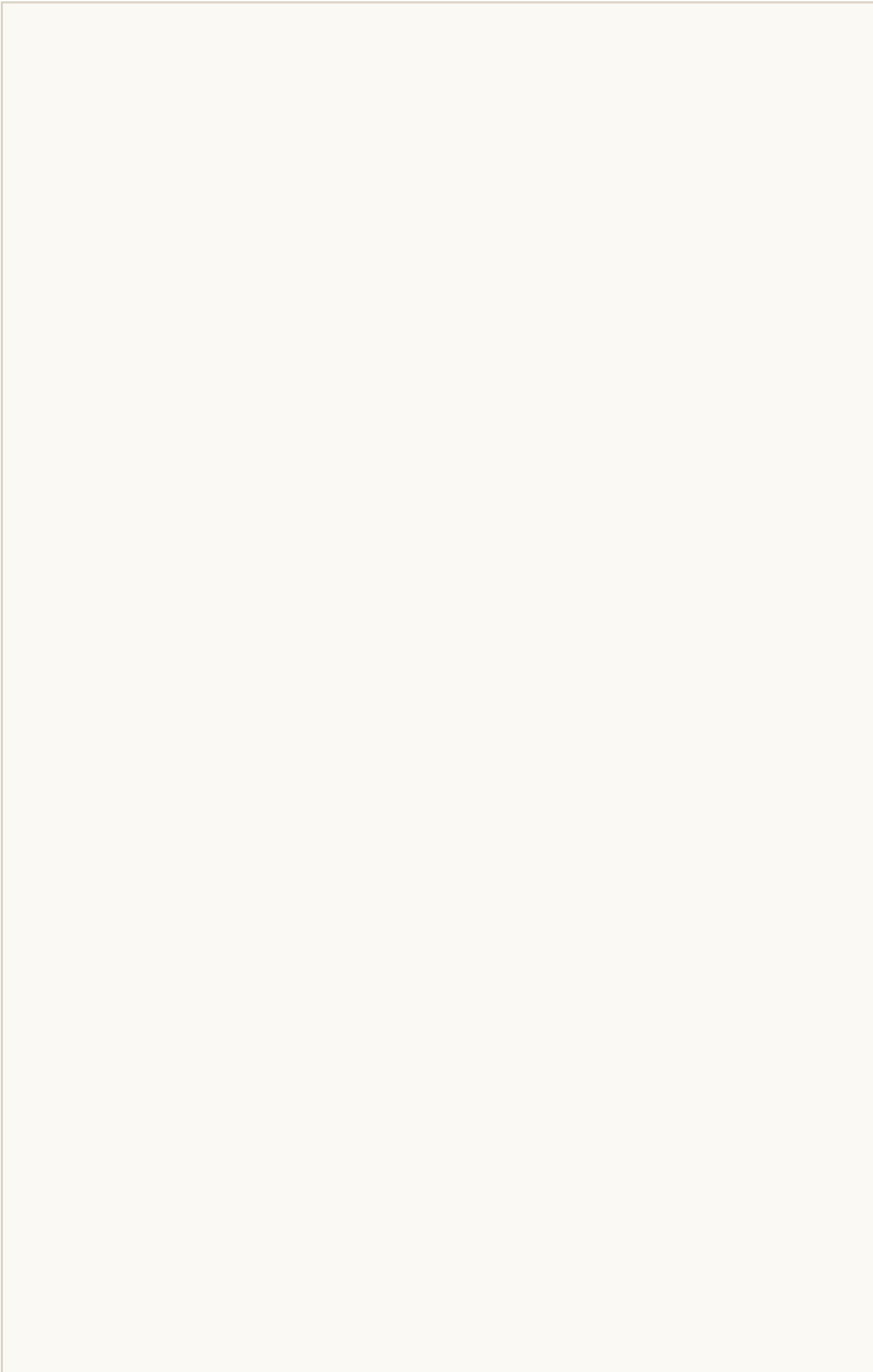
FIG. 2.3 后面一切都架在这个对象上。把鼠标移到任意一个符号上，或者它下面的任意一个短语上，两边会一起亮起来。那顶帽子表示估计出来的，我劝你一直盯着它，因为这一章后面所有的失败，都是这顶帽子到期结账。

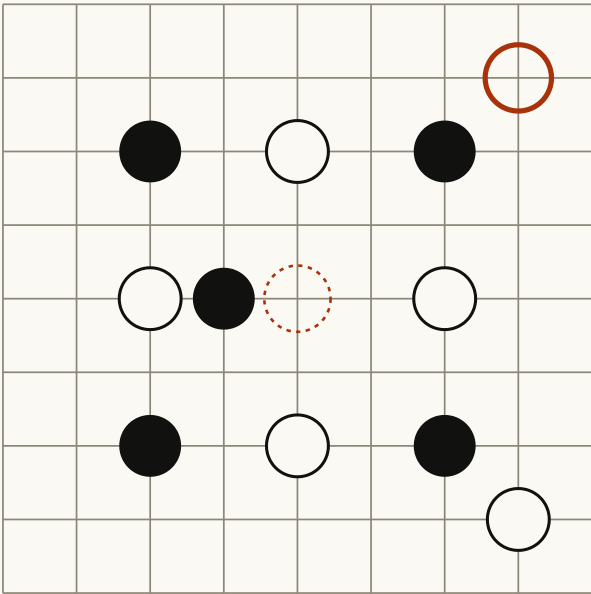
你在哪里这一项，从前是手写下来的。在 1960 年代的控制理论里，也就是 Rudolf Kalman 那个滤波器的世界里（每一台 GPS 接收机里跑的都是这套数学，把阿波罗飞船导过去的也是它），一个东西的状态就是它的位置和速度。2018 年之后的这些年，这个领域一直在放下这个想法，分三步走完，我们一步一步来看。

第一步是 PlaNet。这是 Danijar Hafner 等人 2018 年的系统，写在 *Learning Latent Dynamics for Planning from Pixels* 这篇论文里。它从图像里学出自己的模型，然后就在这个模型里做规划。它从画面里算出自己在哪里，而且从头到尾没给这个答案的任何一部分起过名字。

第二步是 MuZero，它是从外面来的。DeepMind 的 Julian Schrittwieser 等人把它做成了 AlphaGo 与 AlphaZero 的后继者：AlphaGo 赢过人类最强的围棋棋手，AlphaZero 只靠规则就学会了国际象棋、围棋和将棋。而 MuZero 连规则都没拿到。

论文是 2020 年的 *Mastering Atari, Go, chess and shogi by planning with a learned model*。它的模型根本不重建场景。它不产出任何画面，只产出三样说不上是什么的数值：这一步有多好、之后会有多好、该试什么。这三样，每一样头上都戴着帽子。





局面

它可以带上的东西

■ 下一个棋盘的图 240

■ 每颗子在哪儿 90

■ ^ 这有多好 1

■ ^ 该试什么 1

它带着的东西 332

它推荐的一手

■ 下在右上 0.62

下在天元 0.53

它带着的东西	已丢掉	它推荐的一手
332 单位	4 样里的 0 样	下在右上

FIG. 2.4 一个局面，以及模型可以带在身上的四样东西。按一下，把「下一个棋盘的图」丢掉，再把「每颗子在哪儿」丢掉，看它推荐的那一手纹丝不动。反过来，把两个数里的任何一个丢掉，就没有什么可以拿来推荐了。留下来的那两样，头上都戴着帽子。局面和分数仅作参考。

最新的那些系统走出了第三步。它们跑起来完全不带解码器 (网络里负责把压缩过的描述还原成你能看的東西的那一半)。TD-MPC2 就是其中之一，是 Nicklas Hansen 等人 2024 年的系统，用来控制仿真里的机器人。它的论文是 *TD-MPC2: Scalable, Robust World Models for Continuous Control*。

所以模型不必长得像世界，它只需要留住世界做了什么里够用的那一部分。图 2.3 那些符号头上的帽子，就是「这东西是学出来的，而不是像 Kalman 那批工程师那样被人直接交到手上」的代价。这笔代价被收走的那一刻，我后面就管它叫帽子到期结账。

所以「它够不够逼真」是问这类东西时错误的第一个问题。一个画不出国际象棋里那个象的棋局模型，仍然可以是一个非常好的棋局模型。

一次预测本身也不是有用的那部分。给模型一串没有人执行过的动作，让它去读自己的输出，你就得到一段没有人见过的未来。这就是一次推演 (一次想象出来的向前推进：预测，把预测喂回去，再预测)，它的长度叫做视野。



FIG. 2.5 一串没有人执行过的动作。点某一步可以改在那里做什么，拖动视野可以设定这串有多长，然后按「跑一遍」：模型先答第一步，接着把自己刚才的答案读回去，再答第二步。按「留下」可以把这一次钉住，在旁边再写一串。这里什么都没有发生过。数字仅作示意。

我会拿去问每一篇新论文的问题是：你能不能在没人执行过的动作下把它往前推，然后比较结果。画面看起来有多真，和这没关系。Sebastian Thrun 的系统在 1990 年就通过了这个测试，本章末尾他还会再出场。

缓存下来的答案

我把两辆车放在两堵一模一样的墙前面。第一辆在墙近到某个固定距离时刹车。这个距离是某个人当年按预期的速度调好的，调完就发布了。

第二辆带着一个关于自己刹车行为的模型。每一拍它都在问：如果我现在刹，我会停在哪里。在第一辆车被调校的那个速度上，两辆车的行为一模一样，这一点你可以在下面自己验一下。

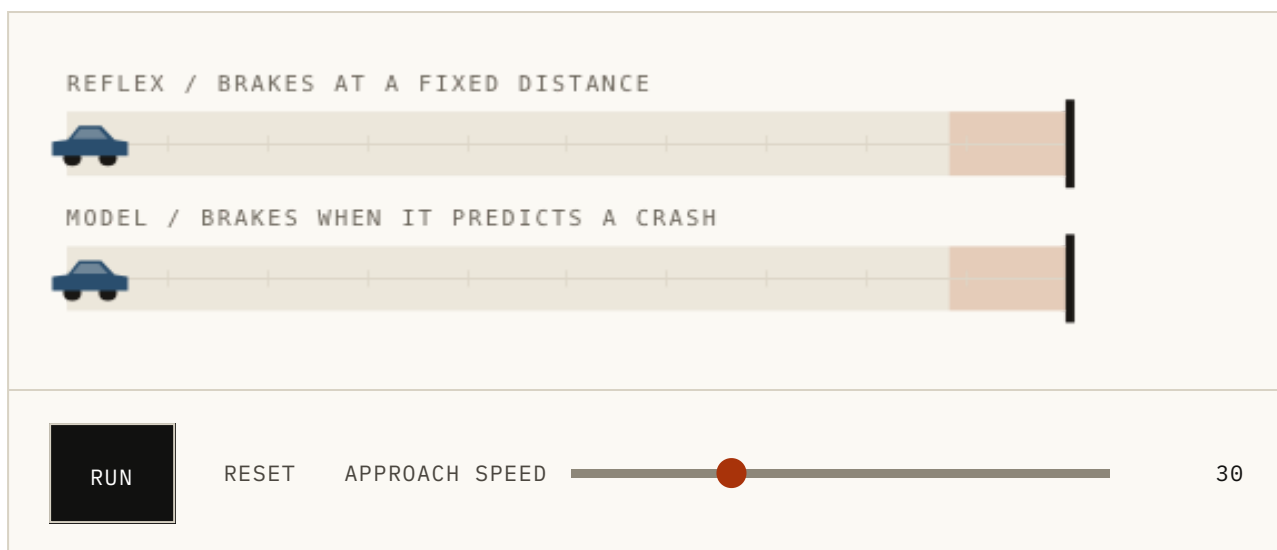


FIG. 2.6 大约在四十以下，两辆车看不出区别。把速度往上拖，看看哪一辆察觉到了。这里比较的是一条固定规则和一个会重新计算的东西。

看到了吗？来得更快一点，触发还是在同一个距离上发生。那条规则里含着一个关于「停下来要多久」的猜测，而它内部没有任何东西知道自己正在猜。模型车之所以改变了它的决定，是因为它的停车点挪了，而它看得见这一点挪动。

这里我得对另一边公平一点。一个真实的、背后没有模型的策略，跟一条固定规则完全不是一回事。它可以是一个记得自己已经看过什么的大网络（循环网络：它把一些东西从每一步交到下一步）。它会根据看到的東西改变自己的行为。

留得住的区别，是思考发生在什么时候。策略在训练阶段就把答案定下了一大半。它留下来的接近于一张答案清单：在这种情形下，做那件事。

模型留下的是另一种条目：在这种情形下，如果你做了那件事，你会得到这个。前者是缓存。后者是等你知道了问题之后，现算一条新条目出来的算法。



FIG. 2.7 一个缓存和一套算法，被问同样的问题。在左边按一个车速，两边都会作答。然后按一个缓存从来没见过：它交回离得最近的那条条目，而算法从头把答案推算出来。顺便看一下两个耗时读数。数字仅作参考。

反射是一个缓存下来的答案。模型是在问题变了以后，让你能算出一个新答案的那个东西。缓存更快，只要你知道自己手里那份什么时候过期了。

缓存通常是对的，这也是为什么生物学和工程学里有那么大一部分是由缓存构成的。麻烦从问题挪出了当初准备答案的那个范围时开始，而你刚才就看着第一辆车这么干了一遍。

这些都不是白来的。模型要花计算时间，要学出来，而且可能是错的。所以真实的系统落在一条连续的谱上，而给这条谱标出两端的那三个系统，前后相隔三十年。

Dyna 是 Richard Sutton 在 1991 年描述的那个智能体，出自 *Dyna: an Integrated Architecture for Learning, Planning and Reacting*。它把真实经验和模型编出来的经验混在一起用，每一步都各来一点。它最后练出来的那个反射，两种都见过。

Dreamer 是 Hafner 2019 年的系统，出自 *Dream to Control* 那篇论文。它在自己的想象里学出一个策略，然后靠反射行动，而 PlaNet 是每一步都要搜一遍。MuZero 站在另一头：一个学出来的模型配上搜索，在定下来之前先在模型里把各种走法试一遍。它们谁都躲不掉的那个选择是：哪些答案提前定好，哪些留到你知道自己面对的是什么之后再算。

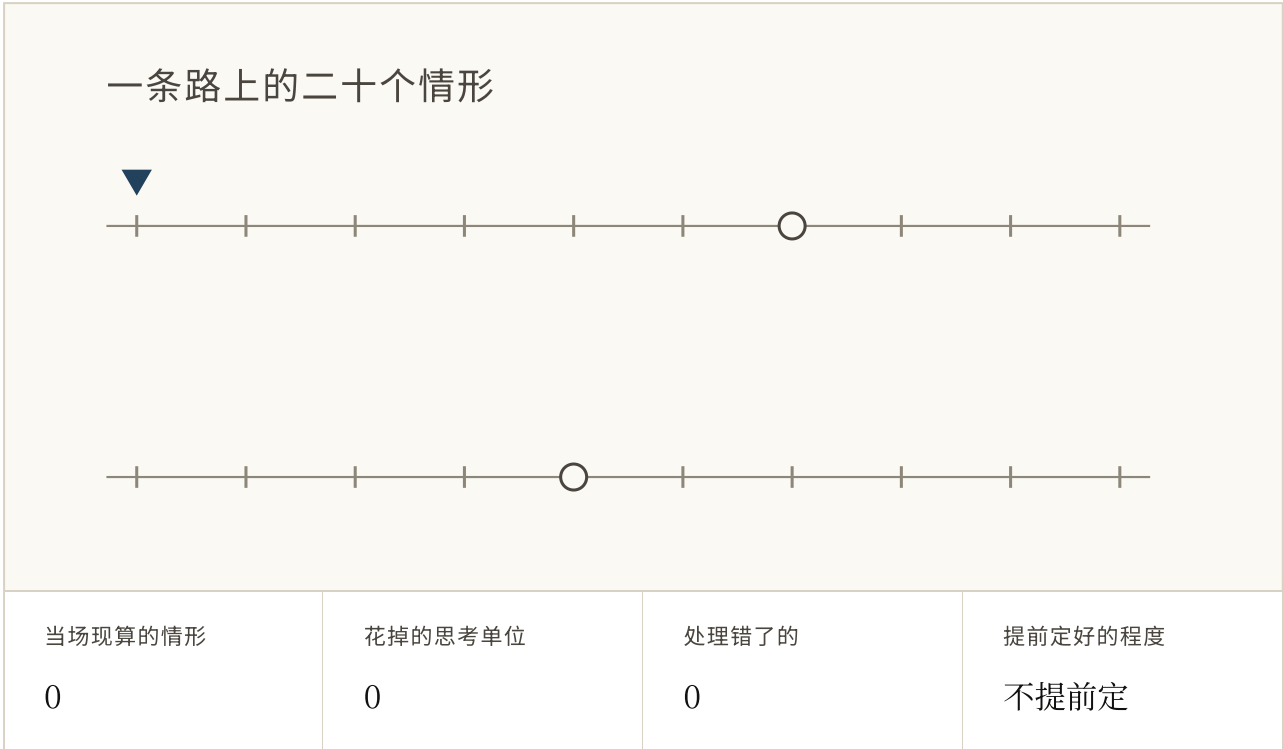


FIG. 2.8 一条路上的二十个情形，只有一个控制项：在见到这条路之前，答案被提前定下了多少。按「开」一步步走，看哪些决定当场就给出，哪些要停下来现算。全都提前定死，二十个里就有两个拿到的是为另一个问题准备好的答案。计数仅作示意。

有了它能换来什么

它能换来三样东西，而三样归根到底都是同一件事：对还没发生的未来动手。我们一样一样来看。

你可以在付出代价之前先试一个动作。把目标放在一个障碍物后面。一个规划器 (在定下来之前，把各种动作先试一遍的那一部分) 写下一串动作，把它放进模型里跑一遍。它给结果打个分，然后扔掉，再写一串。

PlaNet 就是这么一边开一边做的，在它自己对场景的压缩描述里，起点是原始像素。同一年的 *World Models* 那篇论文抢走了注意力，但真正让「从像素做规划」这件事被认真对待的是 PlaNet。下面是这个循环，放慢了看。



FIG. 2.9 按「试一条」，规划器就写下一串动作，放进模型里跑一遍，打个分，然后扔进废纸篓。按「试二十条」可以成批地做。接着按「执行」：目前最好的那一串把它的第一个动作交给世界，剩下的部分也一起进废纸篓。盯着两个步数计数看，因为只有其中一个是要付钱的。仅作参考示意。

只有一个候选时它是在猜，有两百个时它是在挑，而这中间世界什么都没变。所有的提升都来自思考，来自「看见」和「动手」之间的那道缝，而模型就是让这笔交易成为可能的东西。

MuZero 展示了这条路能走多远，而我觉得它到今天仍然是这个领域里最少被人读到的一个结果。它从来没被训练去重画一帧游戏画面或者摆出一个棋盘，它只学了它的搜索会去读的那几个数。它在国际象棋、围棋和将棋上都追平了 AlphaZero，而它被告知的东西还更少。

模型只需要留住这个决定所依赖的东西。那些把逼真画面摆在最前面的人，还没学会这一点。

你可以在犯错很便宜的地方练习。Sutton 的 Dyna 早就在这么做了：从真实经验里学一个模型，再从模型编出来的东西里学。David Ha 与 Schmidhuber 在 2018 年走得更远。他们把自己的控制器完全放在模型对一关 Doom 的梦里训练，然后把它搬回真实游戏，它在那里也站住了。

你想练的那个间隙

在梦里

还没有尝试

在路口

还没有尝试

尝试次数	落在想练的那个间隙上的次数	花掉的时间	险些出事
0	0	0.0 分钟	0

FIG. 2.10 同一个右转，练两遍。把开关留在「在梦里」，按「试一次」：一模一样的那个间隙立刻又来一遍，你想练多少次都行。切到「在路口」再按「试一次」：这回你要等车，间隙从来不是你想要的那个，而且时钟在走。看看有多少次尝试，落在你真正想练的那个间隙上。仅作参考。

一年之后的 Dreamer 把这个把戏放到了正中间。想象出很长的推演，从想象出来的这些里学出行为，等到真要动手的时候已经没什么要现算的了。后面接着发生的事，是这个领域里最接近「王朝」的东西：一个作者，一套配方，六年的版本迭代。

它的第三个版本 DreamerV3 用同一套配方跑了 150 多个任务，没有为其中任何一个重新调参。它是第一个在没有任何人类玩法可抄的情况下，在 Minecraft 里挖到钻石的系统。论文是 2025 年的 *Mastering diverse control tasks through world models*。

很诱人的一句总结是现实很贵，想象免费，而后半句是错的。想象是要花计算时间的，有时候还花掉惊人的量。它省下来的是和真实世界的接触：磨损的机器人、实验室的时间、盯着的人、那次你没法撤销的撞车。模型让你把这笔账的一部分改用计算时间来付。

14 of 32



FIG. 2.11 把更新步数固定住，只拖动这些步是从哪里来的。先用完的预算通常是与真实世界的接触，而算力也仍然是一份预算。一边拖一边看两张账单怎么变。这本账把一个真实步和一个想象步算成同样的价钱，而这个假设正是第 6 章要拆开的。数字仅作示意。

你可以问「如果」。如果我现在刹车会怎样，如果我改成转弯会怎样，如果我从另一边推会怎样。能在不真的执行的前提下比较这些，就是「模型得知道你指的是哪个动作」这件事之所以重要的原因。它把问题从接下来大概会发生什么，变成了如果我做这件事会发生什么。

这个答案在模型内部是真的：以它学到的东西为准，早一点刹车会是这个结果。谁都不该把它错当成一份来自那个并未发生的世界的报告。

一摊没人看见的油、一阵风、任何模型手里从来没有过的东西，都会让这个答案自信地出错。这就是那顶帽子第一次到期结账。



FIG. 2.12 从同一个起点出发的三个「如果」，模型每个都给了答案。按其中一个就是把它执行掉，而世界只回答这一个。你没选的那两个会淡下去，只留下一句「没有人会知道了」，而「重来」给你的是一个新的情形，不是把原来那个还给你。仅作示意。

它在哪里出问题

到这里为止，所有内容都在支持「多来一点」：更好的模型、更多的候选、更长的推演，然后挑出赢家。每一条单看都很合理。它们合在一起，会把你带进这个领域绕着它转的那个失败，而这场复兴在第一年就撞上了它。

Ha 与 Schmidhuber 发现，他们那个在梦里训练出来的控制器，学会了真实游戏根本不允许的作弊招。他们只好把梦调得更随机一些，才把这件事按住。之后 Michael Janner 等人干脆把这个问题写进了 2019 年那篇论文的标题里： *When to Trust Your Model: Model-Based Policy Optimization*。会出问题的是两件不同的事，我们先看作弊这一件。

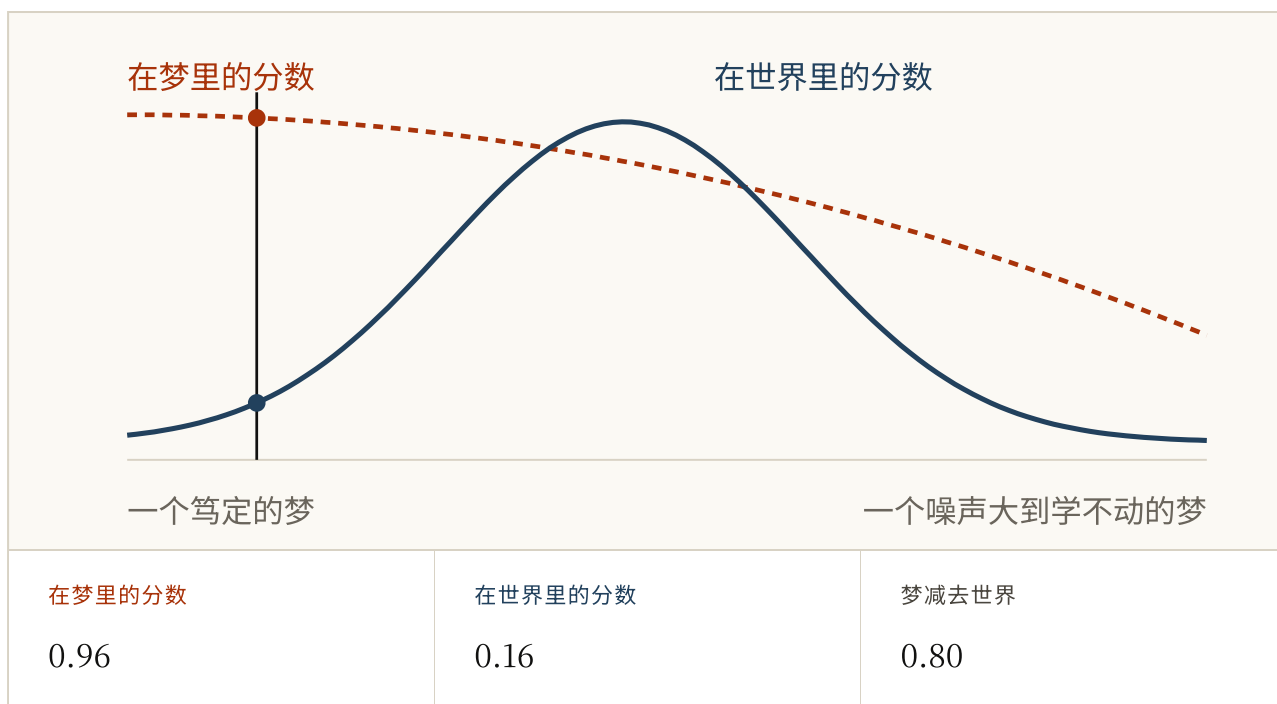


FIG. 2.13 拖动温度，也就是梦有多随机。这些曲线照的是报告出来的形状，不对应某一个具体系统，但结论是 Ha 与 Schmidhuber 的：这个智能体在自己的梦里得分远高于在游戏里，而把不确定性加大会把这个差距合上。要把「好用的那一档」读成一个鼓包，而不是一个方向。

误差会复利。一个只错一次、错一点点的模型没问题。一个错一点点、然后又去读自己那个答案的模型，会有一阵子还行，然后就差得没边了。没有哪一步是坏掉的。

一个单步模型可以准到值得信任，而用同一个模型缝出来的一百步幻想却一文不值。所以「往前看多远才划算」是有上限的，而这也是为什么最粗暴的那个办法，就是干脆让想象出来的推演短一点。

2019 年，Tingwu Wang 等人把主要的基于模型的方法放在一起横向跑了一遍，写成 *Benchmarking Model-Based Reinforcement Learning*。他们给这个上限起了名字，叫规划视野两难，并且把它量了出来。看得太短，规划器看不到目标。看得太长，它就是在照着一个已经漂掉的模型开车。

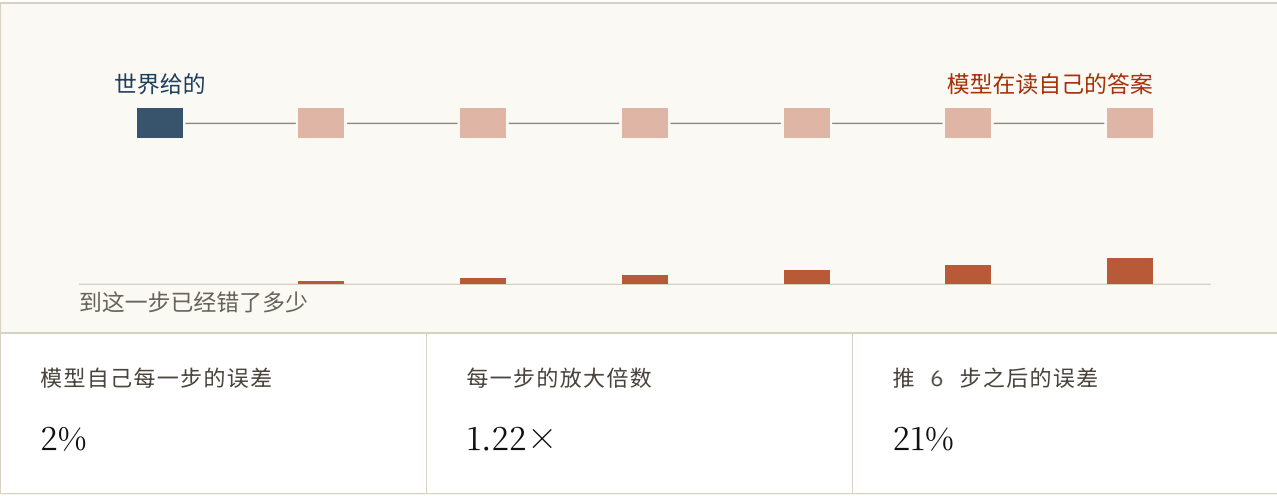


FIG. 2.14 每一步同时发生两件事：模型引入 2% 属于它自己的误差，而已经错掉的部分会被它所穿过的动力学再拉大一截。把放大倍数拖到 1.00，第二件事就关掉了：误差只是单纯累加，柱子排成一条直线。让它大于 1，柱子就开始往上弯。这两个数字都是示意性的；形状不是。

搜索会主动去找那些错误。这一件更古怪，也是我最希望你亲手玩一玩的一件。假设学出来的地图只在一个地方错了：它以为墙上有个缺口，而那面墙是实心的。

随机去试方案，你几乎永远不会靠近那里，弱一点的搜索永远找不到它。彻底一点的搜索会找到，因为在整个「学出来的世界」里，最好的路线正好从那儿穿过去。下面先把力度留在低位，然后再往上调。

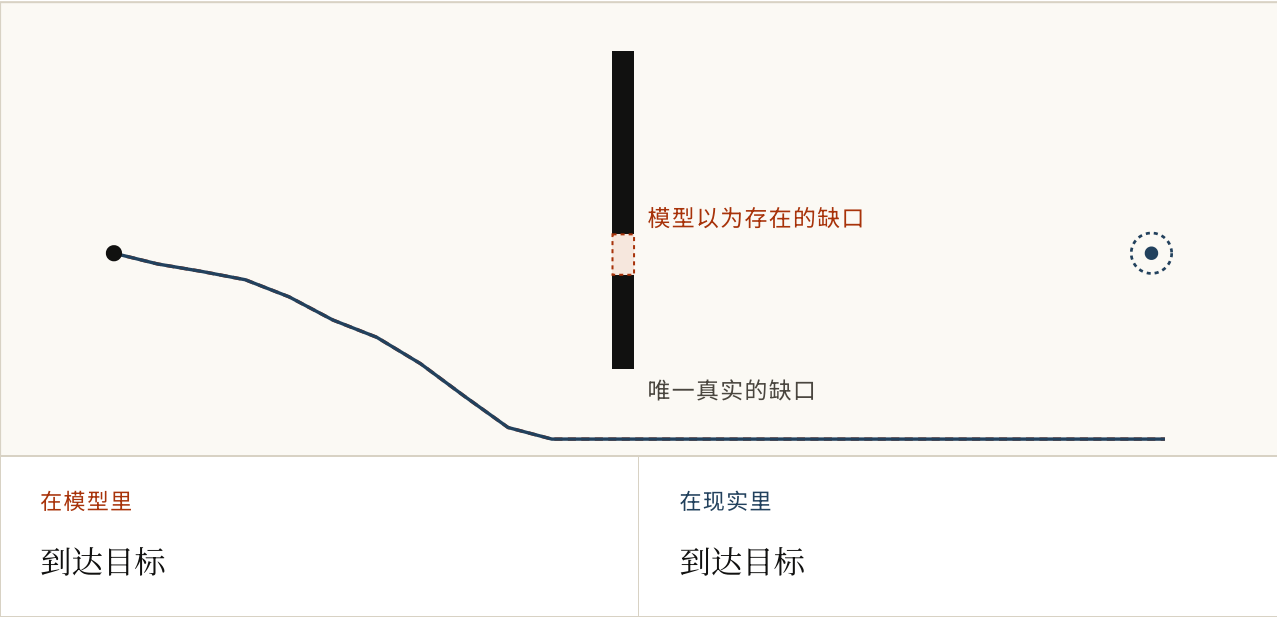


FIG. 2.15 模型相信墙上有个缺口，而并没有。把力度留在低位，规划器根本找不到它，于是老老实实绕远路，然后到达了目标。接着把力度往上调，看着。这是这套设定让你看见的一种失败方式，你不该把它读成「搜索越多结果一定越差」的定律。

这跟大多数工程直觉是反着来的。较弱的搜索给出了一个能用的方案。更强的那个给出了一个分数更高、然后撞墙的方案，而它做的正是你要求它做的事：找出模型最喜欢的那个方案。那个方案，恰好最用力地压在模型朝着对你有利的方向错得最厉害的地方。

这个领域把它叫做**模型利用**。规划器越擅长在模型上拿高分，它就越会往模型从来没被教过的那些地方漂，而那正是它出错的地方。

所以一个平均起来几乎不出错的模型，仍然可能是个危险的、不该在里面做规划的东西。平均误差问的是：在你度量过的那些情况上它表现如何。控制问的是：一旦有东西开始有目的地去搜刮它能找到的最高分，会发生什么。

模型里最好的方案和世界里最好的方案，只有在模型于**搜索会去的地方**也够准时，才会是同一个方案。后面所有的补救办法，都是在设法让最后这半句成立。我先把这个不匹配画成一个形状给你看。

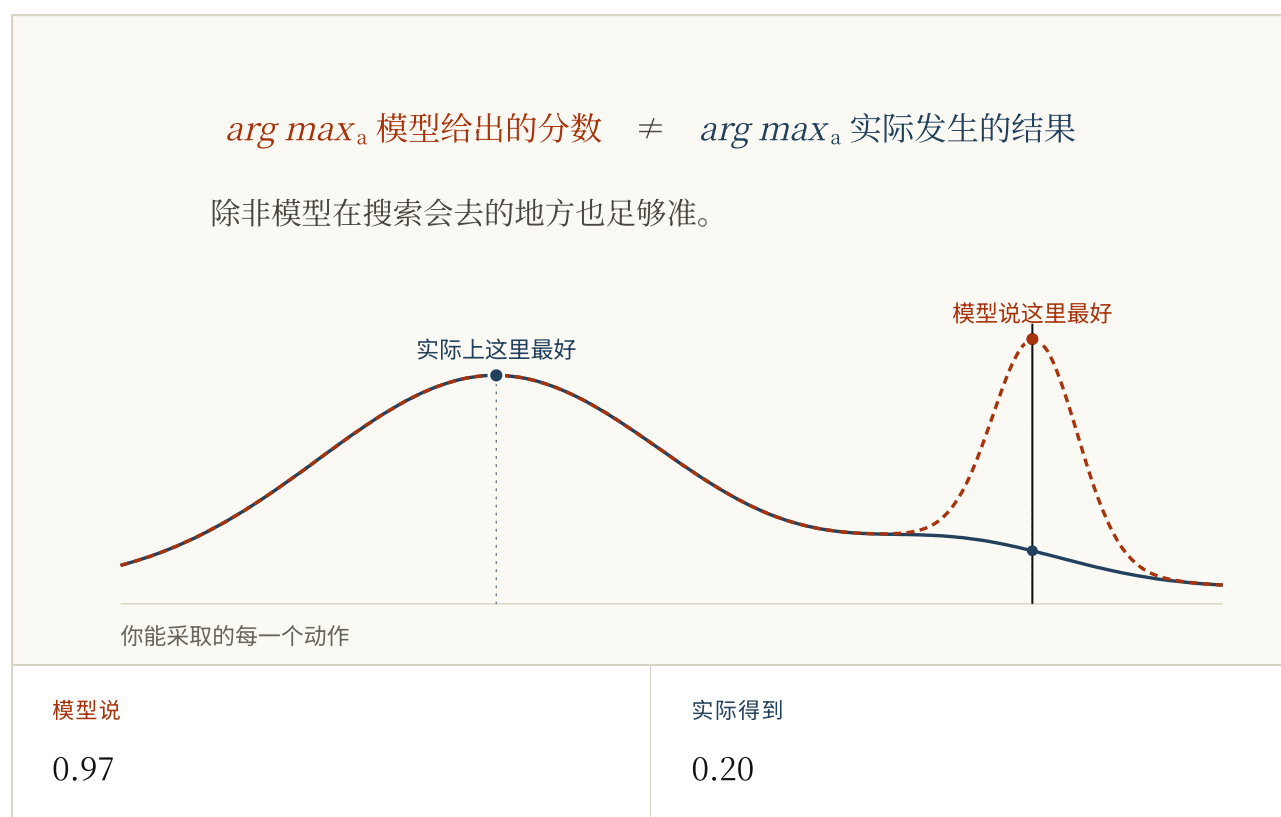


FIG. 2.16 给同一批动作打分的两种方式。它们几乎处处一致，而这正是「平均误差很低」看起来的样子。拖到那个尖峰上，把两个数字都读一遍：模型最中意的那个动作，在想象里得 0.97 分，在真实世界里得 0.20 分。

三十年来都不敢全信这个梦

没有人解决掉这件事，我也不觉得有谁快解决掉了。三十年的基于模型的控制攒下来的，是一整族限制「想象能被信任到多远」的办法。要把它看成一族，得先回到这三十年开始的地方。所以下面是我答应过的那段历史，尽量短。

造词发生在 1990 年，而且不止发生了一次，这通常说明一个想法已经到点了。Schmidhuber 当时在慕尼黑，后来是 LSTM 的共同发明人（LSTM 长期是处理语音和文本的标准网络），他写了一份技术报告 *Making the World Differentiable*，完整标题足足有二十个词。里面一个网络负责预测另一个网络选出的动作会导向什么。他把前一个叫做世界模型。

同一年，Sebastian Thrun 与 Knut Möller、Alexander Linden 一起发表了 *Planning with an Adaptive World Model*。Thrun 更为人所知的身份，是那辆赢下 DARPA 大挑战（一场无人驾驶的沙漠比赛）的斯坦福赛车的作者，以及谷歌自动驾驶项目的发起人。他的系统靠亲自去试，学会自己的动作会带来什么，再把这份知识往前推演，用来挑更好的动作。这个词就印在那篇论文的封面上，比这个标签真正贴住早了二十八年。

SCHMIDHUBER, 1990 <i>Making the World Differentiable</i> 把梯度推过去			
THRUN、MÖLLER 与 LINDEN, 1990 <i>Planning with an Adaptive World Model</i> 在里面搜索			
SUTTON, 1991 <i>Dyna, an Integrated Architecture for Learning, Planning and Reacting</i> 从它那里造出经验			
论文	相隔年数	一个说法的用法	已选
3	1	3	无

FIG. 2.17 三篇论文，一个说法，三种用法。点一张卡片，读一下那篇论文的模型究竟被拿来做什么：把梯度推过去、在里面搜索，还是造出经验来学。然后按「现在谁在做这件事」，每张卡片都会接上本章里做同一件事的那个系统。

Sutton 挖的是同一条矿脉。他和 Andrew Barto 合写了那本至今仍在被用来教强化学习（靠试错和奖励来学习）的教科书，而他的 Dyna 智能体也是这样切分任务的。他还在 Dyna 那篇论文里，写下了把问题讲清楚的那句话。

规划，就是把一个模型作为输入，产出或改进一个用于和被建模的那个世界打交道的策略。

Sutton 谈规划就是在一个学出来的模型上做计算，出自把整个问题立起来的那篇论文（Dyna, an Integrated Architecture for Learning, Planning and Reacting, 1991）
<https://mlanthology.org/icml/1990/sutton1990icml-integrated/>

困难就坐在被建模的那个世界这几个字里，而 Sutton 早在 1991 年就把它放在那儿了，那时候还没有人手里的模型好到足以骗得了人。规划交给你的，是一个对模型里那个世界很好用的策略。那是不是你正站着的这个世界，是另一个问题，而模型内部没有任何东西能回答它。

然后是很长一段时间里没什么动静。神经网络先是过气，到 2010 年代又回来了，而强化学习回来的时候是无模型的：DeepMind 的 Atari 智能体靠试错从原始像素里学，代价是海量的游戏时间。模型就是少付一点的那条路。



FIG. 2.18 这笔账，按天来算。DeepMind 的 Atari 智能体每个游戏大约玩掉五千万帧，折合大约三十八天的游玩时间，而人类测试员拿到的是两小时。选一下这些游玩得发生在哪里，再读一下墙上的钟。然后把模型打开，看这条柱子里真实的那一段缩下去，想象出来的那一段长起来。这两个公开数字是真的；换算仅作示意。

2018 年，Ha 与 Schmidhuber 写了那篇让这个标签流行起来的论文，标题就叫 *World Models*。它是以一个可交互网页的形式发出来的，演示直接在浏览器里跑，而它用更新的零件把 1990 年那个对象搭了出来：一个网络负责看，一个负责预测，一个负责选。他们把负责选的那个叫做控制器，而正如我们在开头看到的，它并不是世界模型。

接下来是那些补救办法。它们出自同样的三十年，读起来像同一条命令，由许多人在相隔几十年的时间里各说了一遍。1991 年 Sutton 的 Dyna 始终有一只脚踩在真实经验里。2018 年 Kurtland Chua 等人的 PETS 同时问好几个模型，并且把它们之间的分歧带着走。2019 年 Janner 让想象出来的推演保持很短，而且都从真实状态出发。2020 年 Tianhe Yu 等人的 MOPO 则把悲观直接建了进去。

多问几个模型。 训练好几个而不是一个 (集成：同样的数据，不同的起点，于是它们在数据从未定下来的地方会互相矛盾)。然后把它们之间的分歧一路带进方案里，就像 PETS 在 2018 年那篇 *Deep RL in a Handful of Trials using Probabilistic Dynamics Models* 里做的那样。

模型们意见不一致的地方，就是方案漂到了没有任何东西支撑的地方。但分歧是个提示，不是判决。在同样的数据空白上训练出来的模型，可能共享同一个盲点，然后自信地在一件根本不存在的事情上达成一致。模型自己的置信度也好不到哪去。

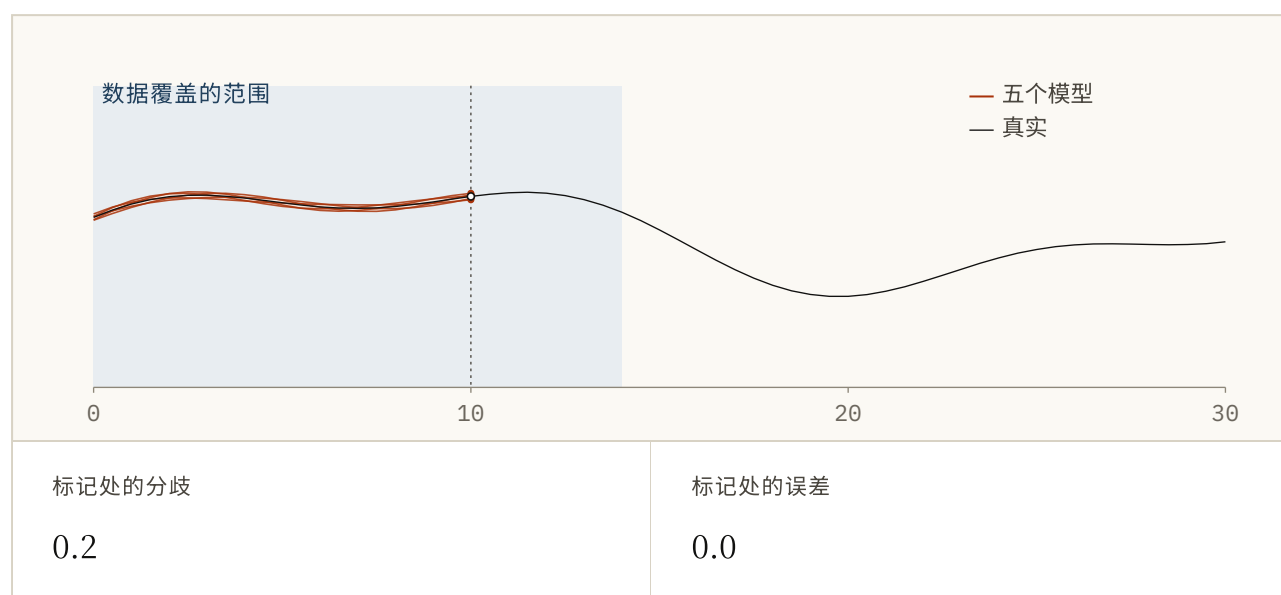


FIG. 2.19 在同一批数据上训练出来的五个模型，从同一个起点往前推演。把计划往右拖，拖过数据结束的地方，看这五个答案怎么散开。那条带子就是它们的分歧。然后把「共同盲区」打开：数据的空白挪了位置，于是五个模型意见一致、靠得很紧，却一起错了。数字仅作示意。

当它说自己有七成把握时，它有七成的时候是对的，前提是真有人去核对过。Ali Malik 等人 2019 年就去核对了，写成 *Calibrated Model-Based Deep Reinforcement Learning*。他们发现它经常不准，也发现把它校正过来之后，规划变

好了。

别做太远的梦。最简单的办法，也是 Janner 的办法，是让每一次想象出来的推演都从世界产生过的某个地方出发，并且保持它很短。你失去了想跑多远就跑多远的自由。你换来的是「一个误差在真实数据插进来之前，能长多久」的上限。

按短时钟做规划也是同一件事。规划一点，执行一点，再看一眼，再规划 (这件事的标准名字叫模型预测控制；它先是在炼油厂里跑，后来才跑到机器人身上，如今机器人学里大半都是它)。重规划没法去掉模型到处都在犯的那种错。但它确实是在不停地把机会交还给世界，让世界来说东西究竟在哪儿。

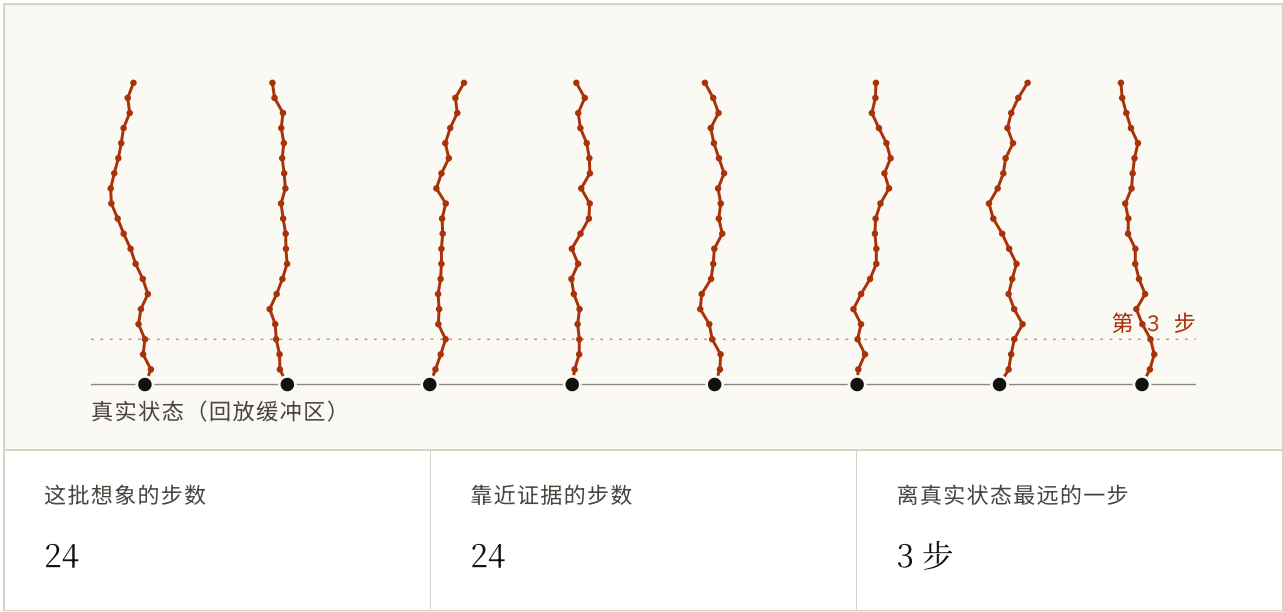


FIG. 2.20 把 Janner 的那笔交易画出来。真实状态排在底部，想象出来的推演从它们身上往上长，每往外一步就淡一点，这代表的是误差在复利。把推演长度往上拖，看分支怎么褪色。然后切到「从一个状态做一条长推演」：同样的长度，想象出来的步数少得多，而且大多数都离任何证据都很远。数字仅作示意。

为「不知道」收费。把模型交回来的分数拿过来，在模型没把握的地方扣掉一块。这样一来，一条看着面生的捷径就必须好到足以抵消「没人知道那底下有什么」的代价。图 2.15 里墙上那个缺口，不再是免费的了。

那些从一堆固定的、录好的经验里学习的系统最需要这个 (叫做离线：没有办法再出去，把你后来发现自己需要的那份数据收回来)。它们没法去拿到那些能纠正自己的经验。而与此同时，规划器要多少时间有多少时间。

所以 Yu 等人 2020 年的 MOPO 才必须为这个离线场景把悲观建进去。论文是 *MOPO: Model-based Offline Policy Optimization*。在它里面，模型许诺的奖励，在模型没把握的地方就要打折。下面你可以看到这对那条捷径做了什么。

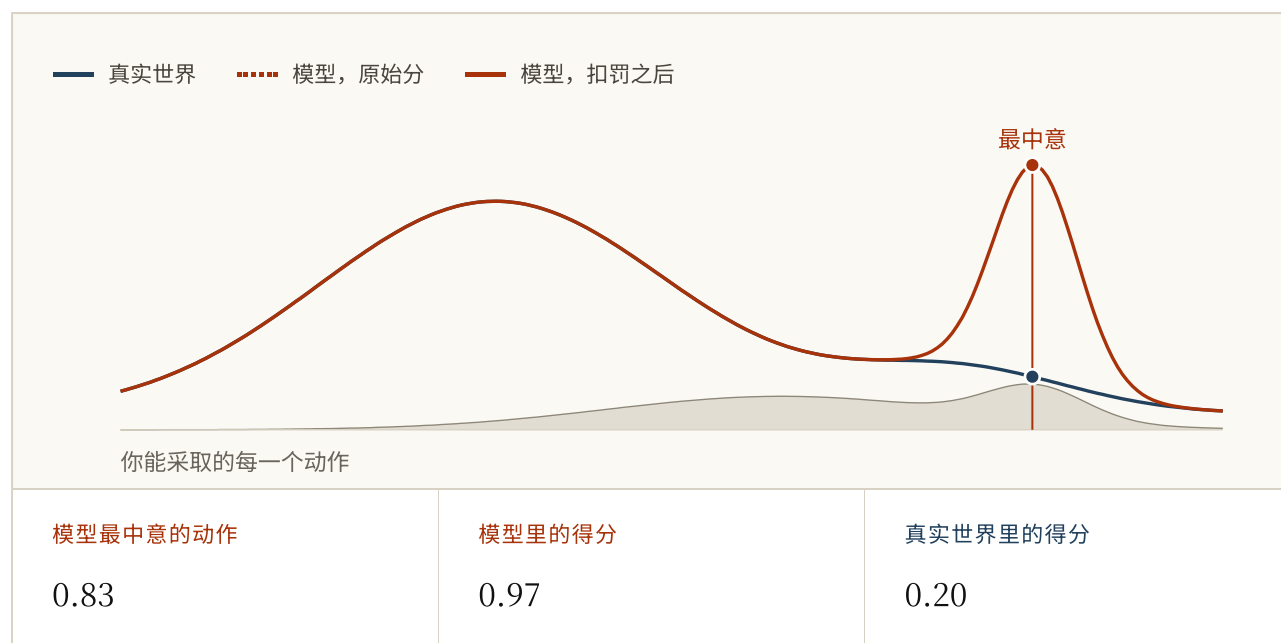


FIG. 2.21 和图 2.16 一样的那两条分数，只是打开了惩罚。把惩罚往上拖，看那个尖峰（也就是模型最没把握的地方）怎么被扣分，一直扣到模型最中意的动作退回到真实世界也认可的那道宽坡上。拖得太过，好动作也会跟着被扣。数字仅作参考。

这三条说的是同一件事，在它们之前的 Dyna 也是。不要让规划器因为跑去了一个模型还没资格自信的地方，而拿到满分。

不管摘要里怎么写，这些都没让问题消失。这些系统里最好的那些，如今能在惊人广的一批任务上站得住：DreamerV3 是 150 多个，TD-MPC2 是 104 个。但它们是小心使用一个错误模型的办法。谁都不该把它们读成「有一个正确的模型」的证明。

光说准不准，几乎什么都没告诉你。你要知道的是在哪里准，对哪些动作准，往前多远还准，面对多强的搜索还准，以及当它不知道的时候，系统该怎么办。而这一切都假设了你手里已经有一个可以往前推的状态：一个位置，一个速度，一小串装着「未来取决

于什么」的数字。可是没有人会把它交给你，你拿到的是像素。

希望这一章对你有帮助。下面有一个小测验，想确认这些想法有没有记住的话可以做一下。如果我哪里写错了，或者你知道比我用的更好的来源，「关于」那一页有一个勘误的部分，我很乐意收到你的来信。

试一试

1 在刹车演示里，速度低于调校值时，固定触发的那辆车和带模型的那辆车行为完全一样。这说明了什么？

- a. 动力学模型什么也没干
- b. 在一条缓存规则当初被准备好的条件范围内，它完全够用
- c. 基于模型的控制只在高速时才有意义
- d. 无模型的系统用不了速度信息

答案 b. 在一条缓存规则当初被准备好的条件范围内，它完全够用。模型并不是到处都赢。在缓存下来的答案仍然正确的那个范围里，重算一遍除了重算的开销之外什么也换不来。

2 一个循环策略把观测和记忆直接映射成动作，但它内部没有一个可以在「没执行过的动作」下往前推的转移函数。它是基于模型的吗？

- a. 是，因为它有记忆
- b. 是，因为任何大网络内部都含着一个模型
- c. 不是
- d. 只有当它的策略是随机的时候才算

答案 c. 不是。记忆和复杂度本身并不构成一个可以调用的动力学模型。缺的那份契约，是在假设性动作下预测后果的能力。

3 某个系统把一帧摄像头画面编码成 512 个数，在这些数上把 500 条候选动作序列各推一遍，然后挑出最好的。它从不解码出未来的图像。它算数吗？

- a. 不算，因为世界模型必须预测像素
- b. 算，因为它学出来的状态可以在动作条件下往前推演
- c. 只有当这 512 个数对应到有名字的物理量时才算
- d. 只有当预测是确定性的时候才算

答案 b. 算，因为它学出来的状态可以在动作条件下往前推演。PlaNet、MuZero 和 TD-MPC2 正是「有用的世界模型必须在视觉上复现未来」这个想法的反例。与决策相关的潜在动力学就够了。

4 Dreamer 用它的世界模型生成的轨迹训练一个 actor，然后由 actor 直接选动作。那一刻并没有大规模搜索在跑。世界模型就不算数了吗？

- a. 不算了，因为规划必须发生在推理时
- b. 还算，因为动力学仍然生成了动作条件下的想象经验
- c. 不算了，除非 actor 把像素重建出来
- d. 这只取决于 actor 有多大

答案 b. 还算，因为动力学仍然生成了动作条件下的想象经验. 在线搜索只是一个可推演动力学模型的用法之一，而不是它的定义。Dreamer 把模型的开销花在更早的训练阶段，而不是动手的那一刻。

5 在模型利用那幅图里，把搜索预算调高之后规划器反而变差了。发生了什么？

- a. 优化器变得不准了
- b. 世界变难了
- c. 更好的优化找到了弱搜索错过的那个模型错误
- d. 长方案总是比短方案差

答案 c. 更好的优化找到了弱搜索错过的那个模型错误. 优化器只是更擅长把模型给出的分数拉高了。错的是那个假设：这个分数在搜索能够到的每一个地方，都还忠实于现实。

6 为什么短模型推演会有帮助？

- a. 神经网络做不了几步以上的预测
- b. 它限制了模型误差和分布偏移在真实数据回来之前能累积多久
- c. 短视野让模型变精确
- d. 它消除了不确定性

答案 b. 它限制了模型误差和分布偏移在真实数据回来之前能累积多久. 这正是 MBPO 那种「从真实状态出发的短推演」的动机：用模型用到足以拿到合成经验，但别用到让累积偏差压倒一切。

7 集成里的五个模型都认为某条捷径是安全的。这证明了什么？

- a. 这条捷径是安全的
- b. 失败的概率为零
- c. 只证明了这五个意见一致；共享的盲点仍然可能存在
- d. 证明了不需要更多模型

答案 c. 只证明了这五个意见一致；共享的盲点仍然可能存在. 分歧是有用的不确定性信号，PETS 用的就是它。但学出来的不确定性本身可能没校准好，在同样有偏的数据上训练出来的模型也可能一起自信地出错。

8 你的动力学模型说：如果你早一秒刹车，你会在撞墙之前停住。你手里拿到的到底是什么？

- a. 物理世界真正会怎么做的证明
- b. 在一个假设性动作下、相对于模型而言的预测
- c. 那条平行历史的录像
- d. 一个与隐藏变量无关的因果结论

答案 b. 在一个假设性动作下、相对于模型而言的预测. 这就是基于模型的控制里「如果」有用的那层意思。它让你在动手之前比较不同动作，而它的权威只延伸到那个学出来的模型所及之处。

FIG. 2.22 八道题，考的是这个智能体在哪里算出后果，而不是它看起来聪不聪明。它们也在问：这份计算什么时候会跑到证据前面去。

参考资料

World Models HA & SCHMIDHUBER, 2018 ↗

编码器、潜在动力学、紧凑控制器，还包括把策略完全放在模型生成的环境里训练、再搬回真实环境的实验。

<https://arxiv.org/abs/1803.10122>

Learning Latent Dynamics for Planning from Pixels (PlaNet)

HAFNER ET AL., 2018 ↗

从图像里学出随机潜在动力学，然后在决策时对它做搜索。图 2.4 的真实版本。

<https://arxiv.org/abs/1811.04551>

Dream to Control (Dreamer) HAFNER ET AL., 2019 ↗

在想象出来的潜在轨迹上训练 actor 和 critic，于是动手的那一刻不必再跑昂贵的搜索。

<https://arxiv.org/abs/1912.01603>

Mastering diverse control tasks through world models (DreamerV3)

HAFNER ET AL., NATURE 2025 ↗

同一个算法、同一套超参数，跑了 150 多个任务。这是想象这一支目前最强的证据。

<https://www.nature.com/articles/s41586-025-08744-2>

Mastering Atari, Go, chess and shogi by planning with a learned model (MuZero)

SCHRITTWIESER ET AL., 2020 ↗

最清楚的一个证明：规划并不需要重建观测。模型只学搜索会消费的那些量。

<https://arxiv.org/abs/1911.08265>

TD-MPC2: Scalable, Robust World Models for Continuous Control

HANSEN, SU & WANG, 2024 ↗

没有解码器的潜在动力学，配上局部轨迹优化，扩展到覆盖 104 个控制任务的单个多任务智能体。

<https://arxiv.org/abs/2310.16828>

Deep RL in a Handful of Trials using Probabilistic Dynamics Models (PETS)

CHUA ET AL., 2018 ↗

集成加轨迹采样：把不确定性做进规划里，而不是当作事后补丁的标准做法。

<https://arxiv.org/abs/1805.12114>

Dyna: an Integrated Architecture for Learning, Planning and Reacting

SUTTON, 1991 ↗

把规划定义成在学出来的模型上做计算，以及那个把它和真实经验交织起来的循环。

<https://mlanthology.org/icml/1990/sutton1990icml-integrated/>

Making the World Differentiable: On Using Self-Supervised Fully Recurrent Neural Networks for Dynamic Reinforcement Learning and Planning in Non-Stationary Environments (FKI-126-90)

SCHMIDHUBER, 1990 ↗

把控制器和世界模型当作两个分开的对象，也就是这一章开头讲的那个区分。

<https://people.idsia.ch/~juergen/world-models-planning-curiosity-fki-1990.html>

Planning with an Adaptive World Model THRUN, MÖLLER & LINDEN, 1990 ↗

通过交互学出一个世界模型，再把它串起来优化未来的动作，比这个标签流行起来早了二十八年。

https://papers.nips.cc/paper_files/paper/1990/hash/9be40cee5b0eee1462c82c6964087ff9-Abstract.html

When to Trust Your Model: Model-Based Policy Optimization

JANNER ET AL., 2019 ↗

从真实状态出发的短推演，直接回应累积误差和模型利用。标题就是这一章的问题。

<https://arxiv.org/abs/1906.08253>

Benchmarking Model-Based Reinforcement Learning WANG ET AL., 2019 ↗

基于模型的方法到底在哪里赢、在哪里输，以及「往前看多远」这个两难在哪里被命名和度量。

<https://arxiv.org/abs/1907.02057>

Calibrated Model-Based Deep Reinforcement Learning MALIK ET AL., 2019 ↗

每一种不确定性方法底下的那句警告：不确定性估计本身可能就是错的，而校正它会改变规划的结果。

<https://arxiv.org/abs/1906.08312>

MOPO: Model-based Offline Policy Optimization YU ET AL., 2020 ↗

把悲观写进目标：按模型的不确定性给预测回报打折，于是面生的捷径必须为「面生」付出代价。

FIG. 2.23 排序是给要在这上面接着做事的人用的，而不是为了历史完整性。